



Silicon Info

A CPU, GPU and Neural Engine widget for Apple Silicon Macs

Documentation, version 1.1

Copyright © 2026 Ray Munro

Free software under the GNU General Public License, version 3 or later.

1. Overview

Silicon Info is a small always-visible panel for the desktop. It breaks Mac activity into the pieces that make up an Apple Silicon chip instead of showing one combined number. The panel has five cards:

- **CPU** performance cores and efficiency cores, shown separately.
- **GPU** device utilization with render and tiler load.
- **Neural Engine** power draw and activity history.
- **Memory** unified memory (DRAM) usage, pressure and swap.
- **Power** how total power divides between the three blocks.

Each card is a summary. Click a card to expand it into a detailed view, and use the Back control to return. The same data is also available as a real macOS desktop widget in three sizes.

2. Requirements

Item	Requirement
Hardware	Mac with Apple Silicon (developed on an M4 Max)
Operating system	macOS 14 or later
Build tools	Xcode and XcodeGen (<code>brew install xcodegen</code>)
Code signing	An Apple Developer team. Widgets and App Groups do not work with ad hoc signing.
Privileges	An administrator password, entered once per launch, for power and frequency data

3. Installation and use

Build

Set your own team in `project.yml` (`DEVELOPMENT_TEAM`), and change the bundle identifiers and the App Group (`875N49PYZ9.com.raymondmunro.siliconinfo` , in `project.yml` and `Sources/Shared/WidgetData.swift`) to match your team. Then:

```
./build.sh
```

This generates the Xcode project from `project.yml` , then builds and signs `SiliconInfo.app` together with its widget extension.

Run

```
open build/Build/Products/Release/SiliconInfo.app
```

The widget appears at the top right of the main display. It has no Dock icon or menu bar item. Drag it anywhere by its background. Its position is remembered, and the window is kept fully on screen when a card expands.

Hide process names

The CPU, GPU and Memory views list process names. To hide those lists, for example before sharing your screen, start the widget with the privacy option:

```
build/Build/Products/Release/SiliconInfo.app/Contents/MacOS/SiliconInfo --hide-processes
```

Quit

```
pkill SiliconInfo
```

4. The views

Overview

Card	Shows
CPU	One bar per core, grouped into performance cores (blue) and efficiency cores (green), each group's average load, package power and a load history line.
GPU	A ring for device utilization, render and tiler bars, power and a history line.
Neural Engine	A ring for power draw and a history line.
Memory	Used and total memory, a bar split into app, wired and compressed memory, and the current pressure level.
Power	A single bar divided between CPU, GPU and Neural Engine, with the total in watts.

CPU detail

Section	Contents
Clusters	For the efficiency cluster and each performance cluster: current frequency, active share, and a histogram of time spent at each clock step.
Cores	Every core with a stacked bar of user time (blue or green) and system time (red), its current clock speed and total load.
Top processes	The busiest processes, as a percentage of one core.
Power	Two minutes of CPU power with average and peak.

GPU detail

Section	Contents
Overview	Utilization ring, current frequency, maximum clock, active and idle share, power.
Frequency residency	Time spent at each GPU clock step.
Engines	Device, render and tiler utilization.
Memory	GPU memory in use and allocated. This memory is shared with the system.
Top GPU processes	Processes using GPU time, as a share of the GPU.

Neural Engine detail

Shows current power, average and peak over two minutes, energy used since launch, an active or idle state and a power history chart. Load is power as a share of an assumed 8 W ceiling, so treat the ring as relative.

Memory detail

Apple Silicon uses one pool of unified memory shared by the CPU, GPU and Neural Engine, so this is the DRAM view. It needs no administrator rights.

Section	Contents
Unified memory	Used, available and total memory, with a bar and a legend for app memory, wired, compressed, cached files and free. Used memory follows the Activity Monitor definition: app plus wired plus compressed.
Pressure and swap	The system pressure level (normal, warning or critical), swap in use and swap size, and the rate of data paged in and out.
Top processes	The processes holding the most resident memory, refreshed every five seconds.
History	Two minutes of memory use.

Power detail

Shows the live split between CPU, GPU and Neural Engine, a stacked two-minute history where the top edge is the total, and average power, peak power and energy in milliwatt-hours for each block since launch.

5. Desktop widget

Silicon Info includes a WidgetKit widget. Run the app once, then right-click the desktop, choose Edit Widgets and search for Silicon Info.

Size	Shows
Small	One component of your choice: CPU, GPU, Neural Engine, Memory or Power. Edit the widget to pick which.
Medium	CPU cores, GPU and Neural Engine side by side, with memory and a power split.
Large	Everything: CPU cores with history, GPU, Neural Engine, memory and power.

How it works

A widget runs in a sandbox and cannot read hardware counters or run `powermetrics`. The Silicon Info app therefore keeps sampling and writes a small snapshot file into an App Group container that both the app and the widget are entitled to use. The widget reads that file whenever macOS redraws it.

macOS limits how often widgets redraw, so expect an update every several seconds, not every second like the floating panel. Keep the app running for fresh data. If it is not running, the widget shows a small note asking you to open it.

6. Data sources

Data	Source	Needs admin
Per-core load, user and system split	Mach <code>host_processor_info</code>	No
GPU utilization, render, tiler, memory	IOKit accelerator statistics	No
Memory breakdown, pressure, swap, paging rates	Mach <code>host_statistics64</code> and <code>sysctl</code>	No
Top processes by memory	<code>ps</code>	No
CPU, GPU and Neural Engine power	<code>powermetrics</code>	Yes
Core and cluster frequency, residency	<code>powermetrics</code>	Yes
GPU frequency and residency	<code>powermetrics</code>	Yes
Per-process CPU and GPU time	<code>powermetrics</code> tasks sampler	Yes

Core order follows Apple Silicon convention: efficiency cores are numbered first, then performance cores.

7. Privileges and safety

Reading power and frequency needs root access. On launch the widget shows the standard macOS administrator dialog. After approval, a small shell loop runs as root. It does only one thing: it runs `powermetrics` once per second and writes the text output to `/private/tmp/siliconwidget-powermetrics.txt`, which the widget reads.

- The loop exits on its own about a second after the last Silicon Info process quits.
- Relaunching quickly reuses the running loop and does not ask again.
- Nothing is sent over the network. The output file contains only power, frequency and process names with CPU and GPU time.
- If you cancel the dialog, the widget keeps running and shows `n/a` for the values that need it.

The output file is readable by other local users, so process names on the machine are visible to them while the widget runs.

8. Limitations

macOS does not expose the following on this hardware, so they are not shown:

- **Neural Engine utilization, frequency or per-process use.** Only power draw is available.

- **Per-cluster CPU power.** Only the CPU total is reported.
- **DRAM bandwidth and DRAM power.** macOS refuses the counter subscription even with administrator rights, so only memory usage is shown.
- **Media engine power.** It reads zero on this chip.

Per-process GPU time is often zero because few processes keep the GPU busy at any moment.

9. Troubleshooting

Symptom	Cause and fix
Power shows <code>n/a</code>	The administrator dialog was cancelled. Quit and relaunch, then approve it.
Power graphs are empty after launch	History fills at one sample per second. Wait a minute.
Neural Engine reads 0 W	The engine is idle. Run something that uses it, such as Core ML work, to see it move.
Widget is off screen	Delete the saved position with <code>defaults delete com.raymondmunro.siliconinfo</code> and relaunch.
Widget says "Open Silicon Info to update"	The app is not running, so no fresh snapshot is being shared. Start the app.
Widget is missing from the gallery	Run the app once. If it still does not appear, copy the app to the Applications folder and open it from there.
Build fails	Install the command line tools with <code>xcode-select --install</code> .

10. Project layout

Path	Purpose
Sources/SiliconInfo/main.swift	Creates the borderless floating window and keeps it anchored and on screen.
Sources/SiliconInfo/Sampler.swift	Samples core load and GPU statistics each second and holds the history.
Sources/SiliconInfo/Memory.swift	Reads unified memory usage, pressure, swap and the top processes by memory.
Sources/SiliconInfo/PowerFeed.swift	Starts the privileged loop and parses the <code>powermetrics</code> output.
Sources/SiliconInfo/Views.swift	The overview cards and expand and collapse behavior.
Sources/SiliconInfo/Detail.swift	The CPU, GPU, Neural Engine, Memory and Power detail views.
Sources/SiliconInfo/WidgetBridge.swift	Builds the snapshot shared with the widget and asks WidgetKit to redraw.
Sources/SiliconInfoWidget/	The WidgetKit extension: timeline provider and widget configuration.
Sources/Shared/	Data model, shared drawing code and the small, medium and large layouts, used by both targets.
project.yml	Xcode project definition, generated by XcodeGen.
scripts/make_icon.swift	Draws the app icon.
build.sh	Generates the project and builds the signed app with its widget.

11. License

Silicon Info is free software: you can redistribute it and modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version. It is distributed in the hope that it will be useful, but without any warranty, without even the implied warranty of merchantability or fitness for a particular purpose. See the LICENSE file for the full text, or <https://www.gnu.org/licenses/>.